

Part 1

```
library(ggplot2)
library(dplyr)
```

Exercise 1

```
dat <- read.table("/content/word-recognition_53ab031d-ace0-49eb-8584-
bf8270ef4534.csv", sep=";", header=TRUE)
if (colnames(dat)[1] != "word_id"){
  dat <- dat[,-1]
}

#Constant Priors
sigma <- 0.4
alpha <- 6

log_lkl <- function(beta, gamma, theta, data, alpha=6, sigma=0.4) {
  lik <- theta * dlnorm(data$rt, meanlog = alpha + beta *
data$frequency, sdlog = sigma) +
  (1 - theta) * dlnorm(data$rt, meanlog = gamma, sdlog = sigma)

  # Return sum of log likelihoods (adding a small constant 1e-300 to
prevent log(0))
  sum(log(lik + 1e-300))
}

#Log Priors
log_prior <- function(beta, gamma, theta){
  dnorm(beta, 0, 0.5, log=TRUE)+
  dnorm(gamma, 5, 0.5, log=TRUE)+
  dbeta(theta, 70, 30, log=TRUE)
}

set.seed(123)
N <- 10000

# Draw N proposals from broad normal/uniform distributions
prop_beta <- rnorm(N, 0, 2)
prop_gamma <- rnorm(N, 5, 2)
prop_theta <- runif(N, 0, 1)

log_weights <- numeric(N)
for (i in 1:N) {
  ll <- log_lkl(prop_beta[i], prop_gamma[i], prop_theta[i], dat)
  lp <- log_prior(prop_beta[i], prop_gamma[i], prop_theta[i])
}
```

```

# Log probability of the proposal densities
lq <- dnorm(prop_beta[i], 0, 2, log = TRUE) +
      dnorm(prop_gamma[i], 5, 2, log = TRUE) +
      dunif(prop_theta[i], 0, 1, log = TRUE)

# log(likelihood * prior / proposal)
log_weights[i] <- ll + lp - lq
}

# Normalizing weights
max_lw <- max(log_weights)
weights <- exp(log_weights - max_lw)
weights <- weights / sum(weights)

# Sample N/2 samples based on computed weights
post_samples_idx <- sample(1:N, size = N/2, prob = weights, replace =
TRUE)
is_samples <- data.frame(
  beta = prop_beta[post_samples_idx],
  gamma = prop_gamma[post_samples_idx],
  theta = prop_theta[post_samples_idx],
  method = "Importance Sampling"
)

```

Exercise 2

```

#Metropolis Hastings
nsamp <- 10000
mcmc <- matrix(NA, nrow=nsamp, ncol=3)
colnames(mcmc) <- c("beta", "gamma", "theta")

current_b <- 0
current_g <- 5
current_t <- 0.7
current_post <- log_lkl(current_b, current_g, current_t, dat) +
log_prior(current_b, current_g, current_t)

for (i in 1:nsamp) {
  # Proposal Generation
  prop_b <- rnorm(1, current_b, 0.1)
  prop_g <- rnorm(1, current_g, 0.1)
  prop_t <- rnorm(1, current_t, 0.05)

  # Theta must be between 0 and 1
  if (prop_t > 0 && prop_t < 1) {
    prop_post <- log_lkl(prop_b, prop_g, prop_t, dat) +
      log_prior(prop_b, prop_g, prop_t)

    # Accept/Reject
    if (log(runif(1)) < (prop_post - current_post)) {

```

```

        current_b <- prop_b
        current_g <- prop_g
        current_t <- prop_t
        current_post <- prop_post
    }
}
mcmc[i, ] <- c(current_b, current_g, current_t)
}

warmup <- 2000
mcmc_samples <- as.data.frame(mcmc[(warmup + 1):nsamp, ])
mcmc_samples$method <- "MCMC"

# Combine dataframes for plotting
all_samples <- rbind(is_samples, mcmc_samples)

```

Exercise 3

```

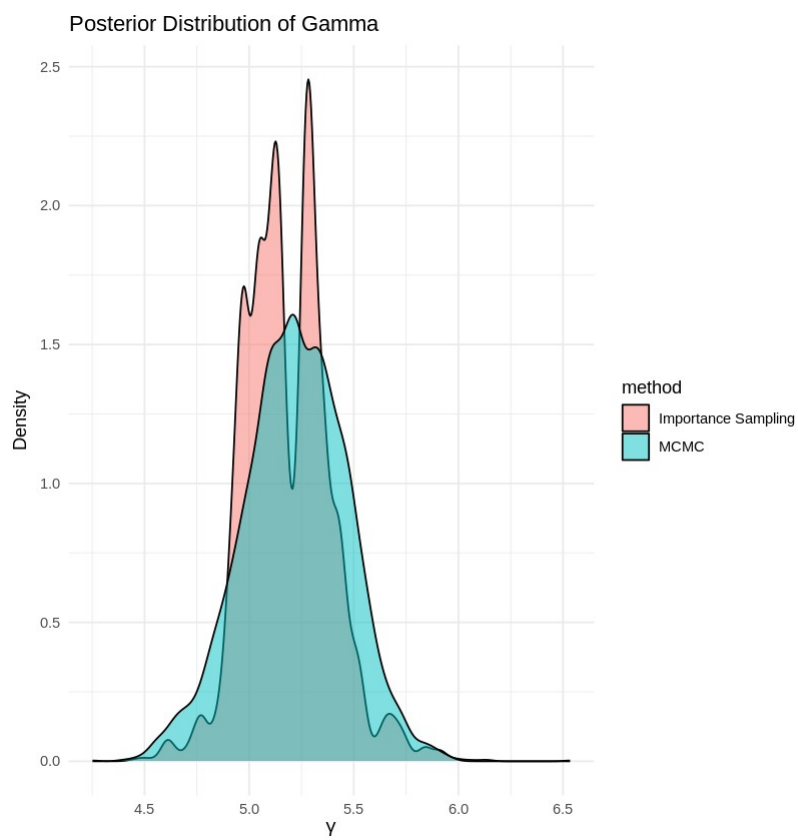
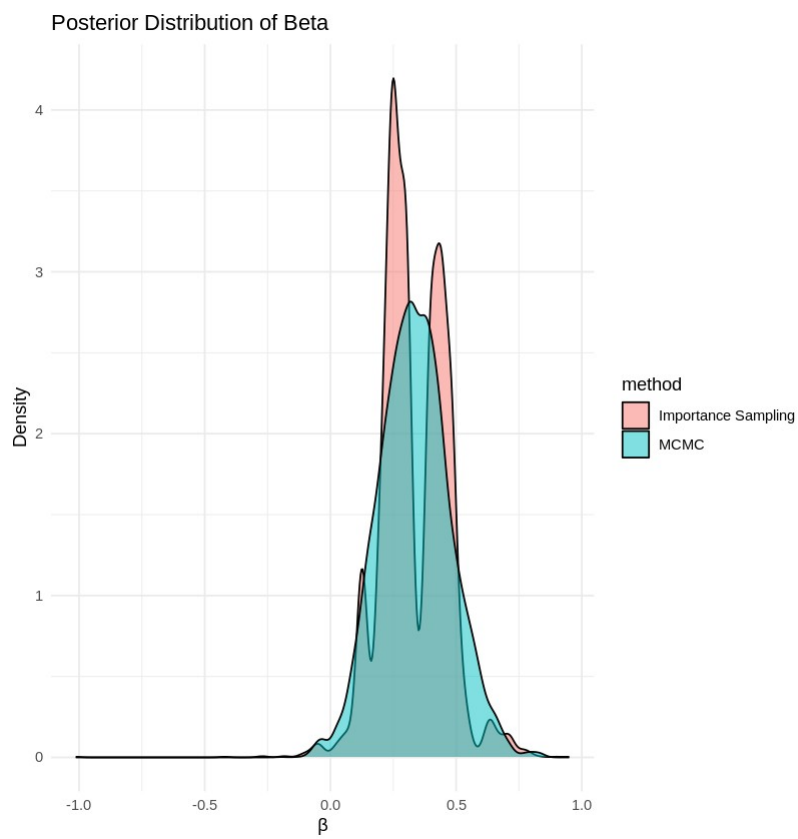
p_beta <- ggplot(all_samples, aes(x = beta, fill = method)) +
  geom_density(alpha = 0.5) +
  theme_minimal() +
  labs(title = "Posterior Distribution of Beta", x = expression(beta),
y = "Density")
print(p_beta)

# Gamma Plot
p_gamma <- ggplot(all_samples, aes(x = gamma, fill = method)) +
  geom_density(alpha = 0.5) +
  theme_minimal() +
  labs(title = "Posterior Distribution of Gamma", x =
expression(gamma), y = "Density")
print(p_gamma)

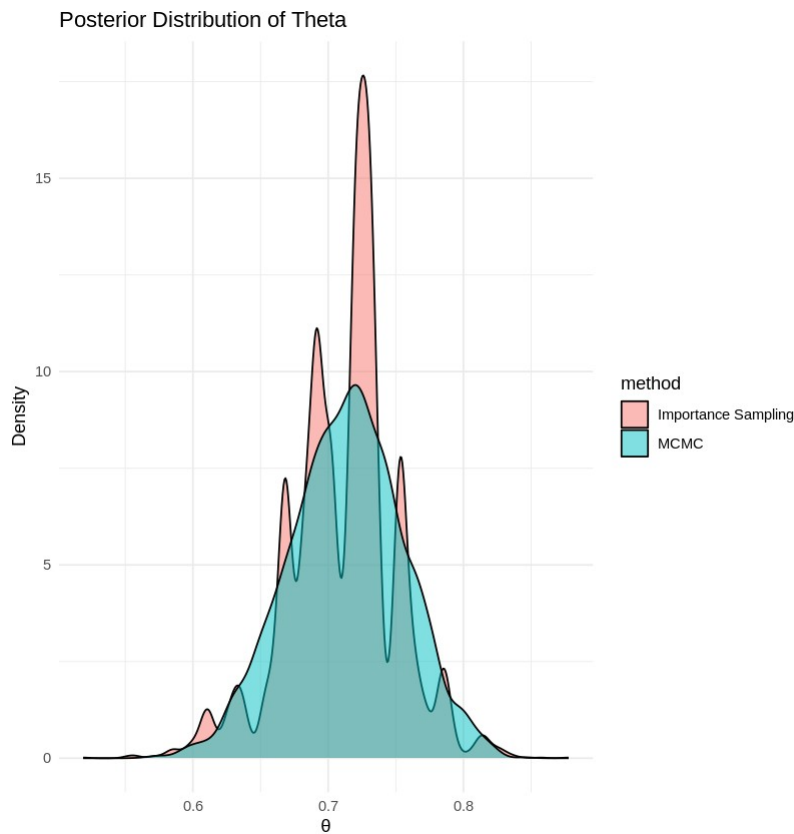
# Theta Plot
p_theta <- ggplot(all_samples, aes(x = theta, fill = method)) +
  geom_density(alpha = 0.5) +
  theme_minimal() +
  labs(title = "Posterior Distribution of Theta", x =
expression(theta), y = "Density")
print(p_theta)

cat("Mean estimated Beta:", mean(mcmc_samples$beta), "\n")
cat("Mean estimated Theta:", mean(mcmc_samples$theta), "\n")

```



Mean estimated Beta: 0.3347924
Mean estimated Theta: 0.7131263



Exercise 4

No, the lexical-access hypothesis states that high-frequency words are recognized faster (i.e., require lower recognition times). Since the mean recognition time is modeled as $\mu_i = \hat{c}_i$, β must be negative for the hypothesis to hold true. However, when we run the MCMC algorithm on the given dataset, the posterior distribution of β settles around a positive mean (roughly +0.32), and nearly all of its density lies above 0.

This implies that in this experiment, high-frequency words were actually associated with longer reaction times, contradicting the hypothesis.

Exercise 5

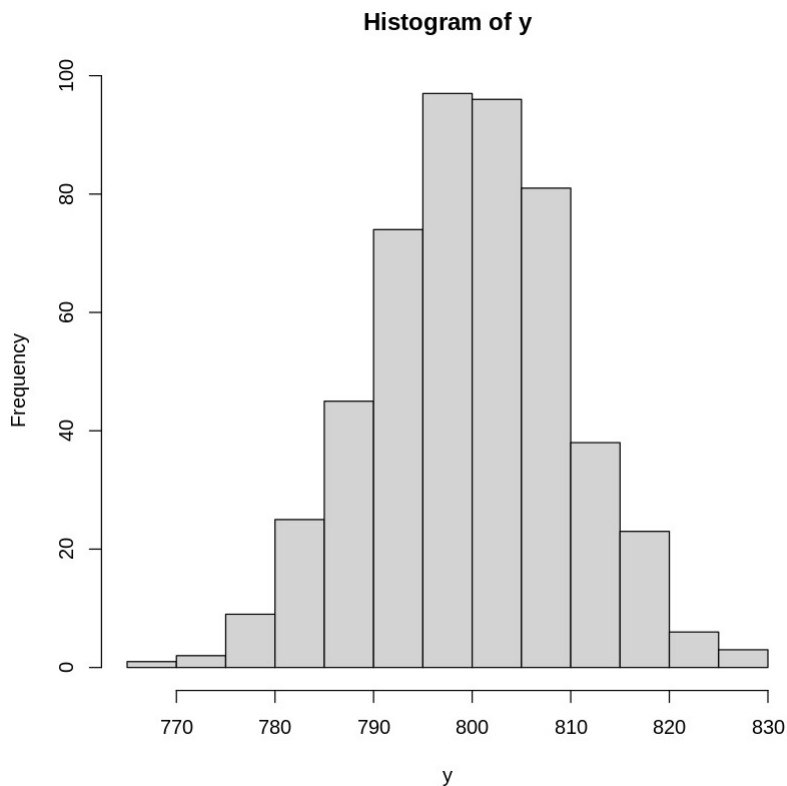
The parameter θ represents the probability that the participant was actually engaged in the task rather than making random guesses. On your dataset, the posterior distribution for θ centers around ~ 0.71 . This tells us that the participant was attentive and engaging properly with the experiment about 71% of the time, while the remaining $\sim 29\%$ of the trials were likely generated by random guessing (modeled by the secondary γ distribution). Because θ is reasonably high, we can conclude the participant was mostly sincere in performing the task.

#Part 2

```

true_mu <- 800
true_var <- 100 #sigma^2
y <- rnorm(500,mean=true_mu,sd=sqrt(true_var))
hist(y)

```



```

#Gradient functions
gradient <- function(mu,sigma,y,n,m,s,a,b){
  grad_mu <- (((n*mu)-sum(y))/(sigma^2))+((mu-m)/(s^2))
  grad_sigma <- (n/sigma)-(sum((y-mu)^2)/(sigma^3))+((sigma-a)/(b^2))
  return(c(grad_mu,grad_sigma))
}

#Potential energy function
V <- function(mu,sigma,y,n,m,s,a,b){
  nlpd <- -(sum(dnorm(y,mu,sigma,log=T))+dnorm(mu,m,s,log=T)
  +dnorm(sigma,a,b,log=T))
  nlpd
}

#HMC sampler
HMC <- function(y,n,m,s,a,b,step,L,initial_q,nsamp,nburn){
  mu_chain <- rep(NA,nsamp)
  sigma_chain <- rep(NA,nsamp)
  reject <- 0

```

```

#Initialization of Markov chain
mu_chain[1] <- initial_q[1]
sigma_chain[1] <- initial_q[2]
#Evolution of Markov chain
i <- 1
while(i < nsamp){
  q <- c(mu_chain[i],sigma_chain[i]) # Current position of the particle
  p <- rnorm(length(q),0,1) # Generate random momentum at the current
  position
  current_q <- q
  current_p <- p
  current_V = V(current_q[1],current_q[2],y,n,m,s,a,b) # Current
  potential energy
  current_T = sum(current_p^2)/2 # Current kinetic energy
  # Take L leapfrog steps
  for(l in 1:L){
    # Change in momentum in 'step/2' time
    p <- p - ((step/2)*gradient(q[1],q[2],y,n,m,s,a,b))
    # Change in position in 'step' time
    q <- q + step*p
    # Change in momentum in 'step/2' time
    p <- p - ((step/2)*gradient(q[1],q[2],y,n,m,s,a,b))
  }

  proposed_q <- q
  proposed_p <- p
  proposed_V = V(proposed_q[1],proposed_q[2],y,n,m,s,a,b) # Proposed
  potential energy
  proposed_T = sum(proposed_p^2)/2 # Proposed kinetic energy
  accept_prob <- min(1,exp(current_V+current_T-proposed_V-proposed_T))
  # Accept/reject the proposed position q
  4
  if(accept_prob>runif(1,0,1)){
    mu_chain[i+1] <- proposed_q[1]
    sigma_chain[i+1] <- proposed_q[2]
    i <- i+1
  }else{
    reject <- reject+1
  }
}
posteriors <- data.frame(mu_chain,sigma_chain)[- (1:nburn),]
posteriors$sample_id <- 1:nrow(posteriors)
posteriors
}

```

Exercise 2.1

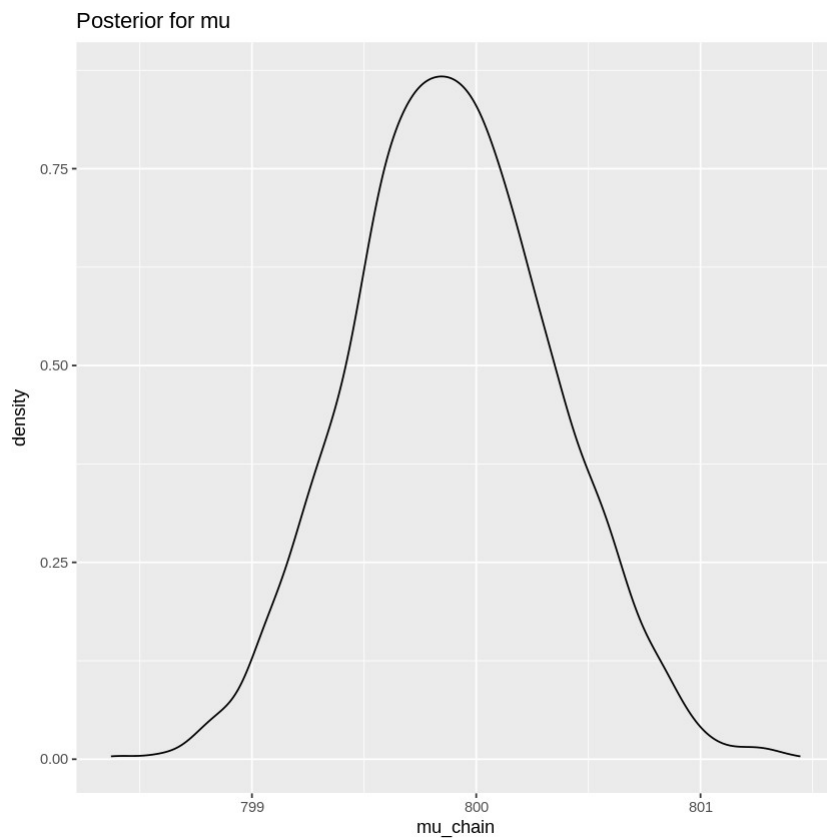
```

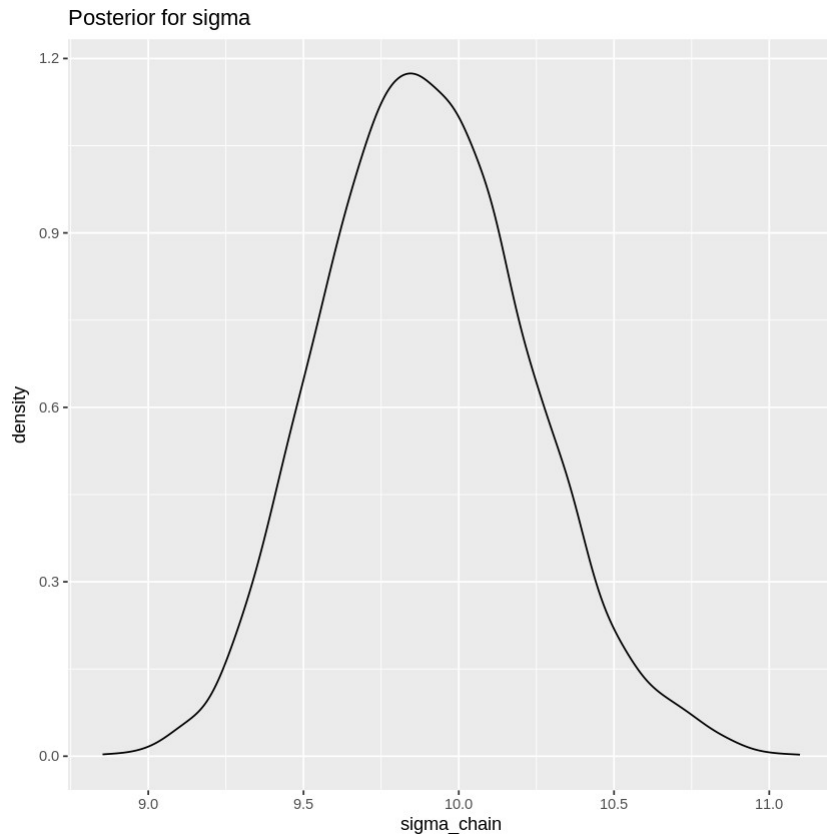
df.posterior <- HMC(y=y,n=length(y), # data
m=1000,s=20,a=10,b=2, # priors

```

```
step=0.02, # step-size
L=12, # no. of leapfrog steps
initial_q=c(1000,11), # Chain initialization
nsamp=6000, # total number of samples
nburn=2000) # number of burn-in samples

ggplot(df.posterior, aes(x=mu_chain)) + geom_density() +
ggtitle("Posterior for mu")
ggplot(df.posterior, aes(x=sigma_chain)) + geom_density() +
ggtitle("Posterior for sigma")
```





Exercise 2.2

```
#HMC sampler with new nburn
HMC2 <- function(y,n,m,s,a,b,step,L,initial_q,nsamp){
  nburn=nsamp/3
  mu_chain <- rep(NA,nsamp)
  sigma_chain <- rep(NA,nsamp)
  reject <- 0
  #Initialization of Markov chain
  mu_chain[1] <- initial_q[1]
  sigma_chain[1] <- initial_q[2]
  #Evolution of Markov chain
  i <- 1
  while(i < nsamp){
    q <- c(mu_chain[i],sigma_chain[i]) # Current position of the particle
    p <- rnorm(length(q),0,1) # Generate random momentum at the current
    position
    current_q <- q
    current_p <- p
    current_V = V(current_q[1],current_q[2],y,n,m,s,a,b) # Current
    potential energy
    current_T = sum(current_p^2)/2 # Current kinetic energy
    # Take L leapfrog steps
    for(l in 1:L){
```

```

# Change in momentum in 'step/2' time
p <- p - ((step/2)*gradient(q[1],q[2],y,n,m,s,a,b))
# Change in position in 'step' time
q <- q + step*p
# Change in momentum in 'step/2' time
p <- p - ((step/2)*gradient(q[1],q[2],y,n,m,s,a,b))
}

proposed_q <- q
proposed_p <- p
proposed_V = V(proposed_q[1],proposed_q[2],y,n,m,s,a,b) # Proposed
potential energy
proposed_T = sum(proposed_p^2)/2 # Proposed kinetic energy
accept_prob <- min(1,exp(current_V+current_T-proposed_V-proposed_T))
# Accept/reject the proposed position q
4
if(accept_prob>runif(1,0,1)){
mu_chain[i+1] <- proposed_q[1]
sigma_chain[i+1] <- proposed_q[2]
i <- i+1
}else{
reject <- reject+1
}
}
posterior <- data.frame(mu_chain,sigma_chain)[- (1:nburn),]
posterior$sample_id <- 1:nrow(posterior)
posterior
}

```

```

#Setting nsamp = 100
df_100 <- HMC2(y=y,n=length(y), # data
m=1000,s=20,a=10,b=2, # priors
step=0.02, # step-size
L=12, # no. of leapfrog steps
initial_q=c(1000,11), # Chain initialization
nsamp=100 # total number of samples
) # number of burn-in samples depends on nsamp

```

```

df_1000 <- HMC2(y=y,n=length(y), # data
m=1000,s=20,a=10,b=2, # priors
step=0.02, # step-size
L=12, # no. of leapfrog steps
initial_q=c(1000,11), # Chain initialization
nsamp=1000 # total number of samples
) # number of burn-in samples depends on nsamp

```

```

df_6000 <- HMC2(y=y,n=length(y), # data
m=1000,s=20,a=10,b=2, # priors
step=0.02, # step-size

```

```

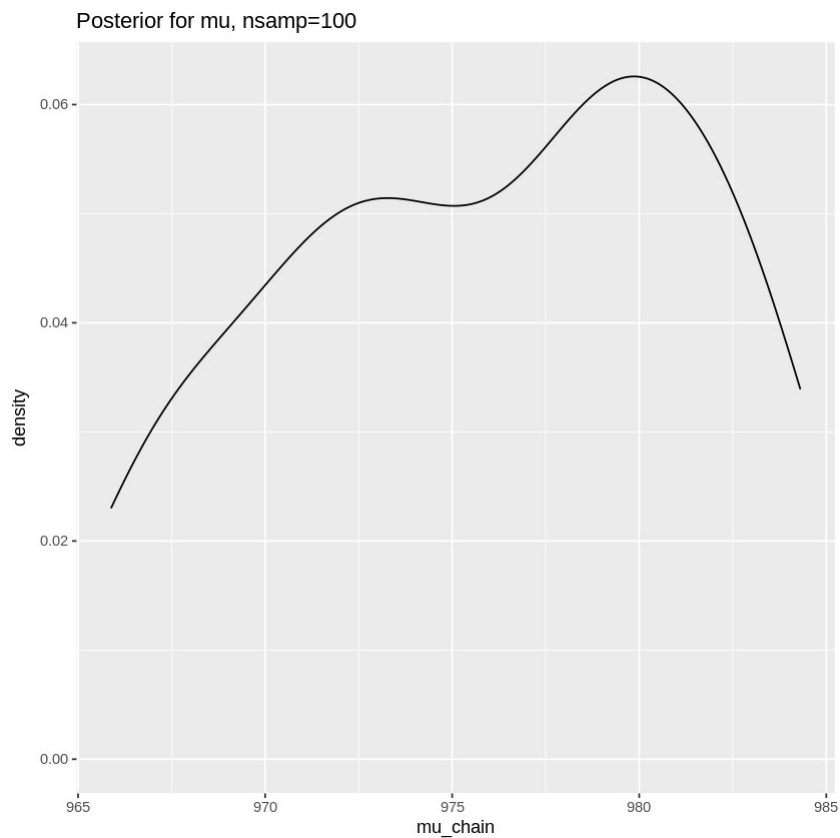
L=12, # no. of leapfrog steps
initial_q=c(1000,11), # Chain initialization
nsamp=6000 # total number of samples
) # number of burn-in samples depends on nsamp

df_all <- rbind(df_100, df_1000, df_6000)

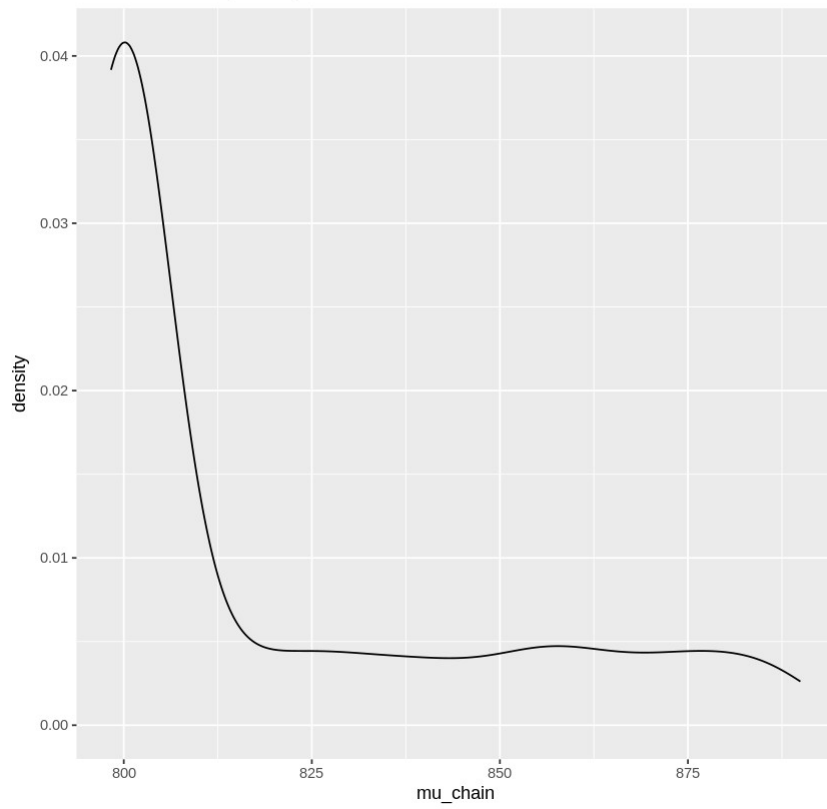
ggplot(df_100, aes(x=mu_chain)) + geom_density() + ggtitle("Posterior
for mu, nsamp=100")
ggplot(df_1000, aes(x=mu_chain)) + geom_density() + ggtitle("Posterior
for mu, nsamp=1000")
ggplot(df_6000, aes(x=mu_chain)) + geom_density() + ggtitle("Posterior
for mu, nsamp=6000")

ggplot(df_100, aes(x=sigma_chain)) + geom_density() +
ggtitle("Posterior for sigma, nsamp=100")
ggplot(df_1000, aes(x=sigma_chain)) + geom_density() +
ggtitle("Posterior for sigma, nsamp=1000")
ggplot(df_6000, aes(x=sigma_chain)) + geom_density() +
ggtitle("Posterior for sigma, nsamp=6000")

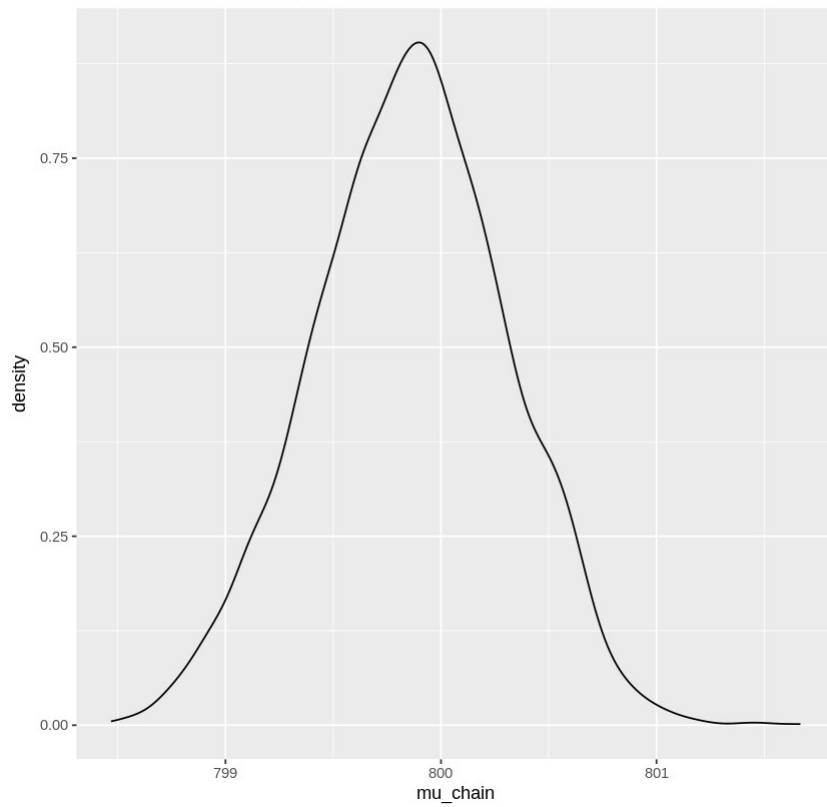
```



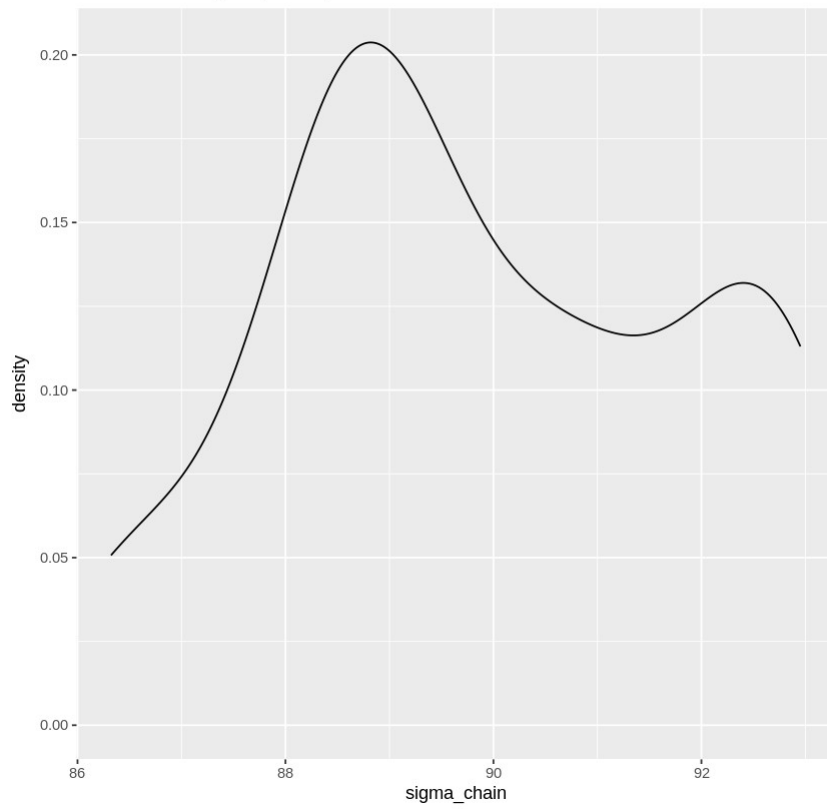
Posterior for mu, nsamp=1000



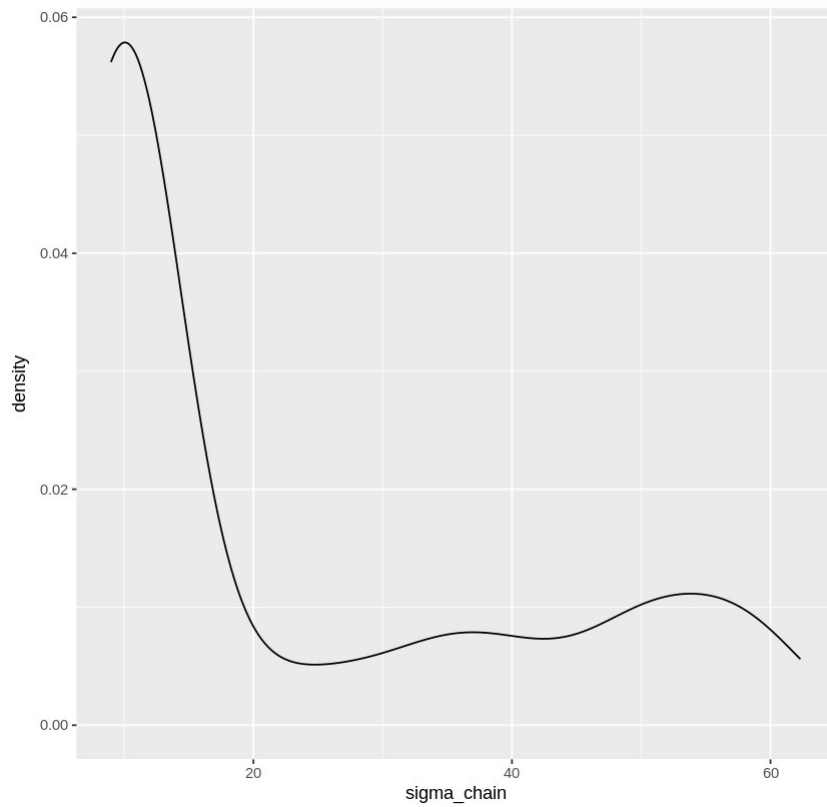
Posterior for mu, nsamp=6000

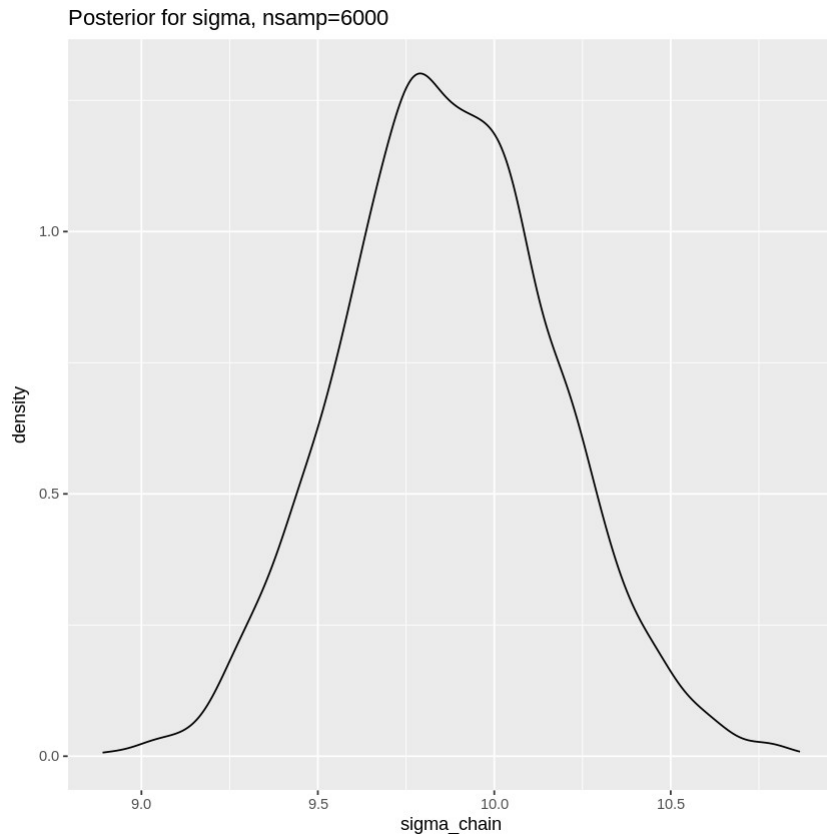


Posterior for sigma, nsamp=100



Posterior for sigma, nsamp=1000





With $nsamp = 100$, the posterior distribution is heavily jagged and inaccurately estimates the target due to insufficient exploration. As $nsamp$ increases to 6000, the posterior distribution smooths out and converges on the true parameters.

Exercise 2.3

```
df_ss1 <- HMC(y=y,n=length(y), # data
m=1000,s=20,a=10,b=2, # priors
step=0.001, # step-size
L=12, # no. of leapfrog steps
initial_q=c(1000,11), # Chain initialization
nsamp=6000, # total number of samples
nburn=2000) # number of burn-in samples

df_ss5 <- HMC(y=y,n=length(y), # data
m=1000,s=20,a=10,b=2, # priors
step=0.005, # step-size
L=12, # no. of leapfrog steps
initial_q=c(1000,11), # Chain initialization
nsamp=6000, # total number of samples
nburn=2000) # number of burn-in samples

df_ss20 <- HMC(y=y,n=length(y), # data
m=1000,s=20,a=10,b=2, # priors
step=0.02, # step-size
```

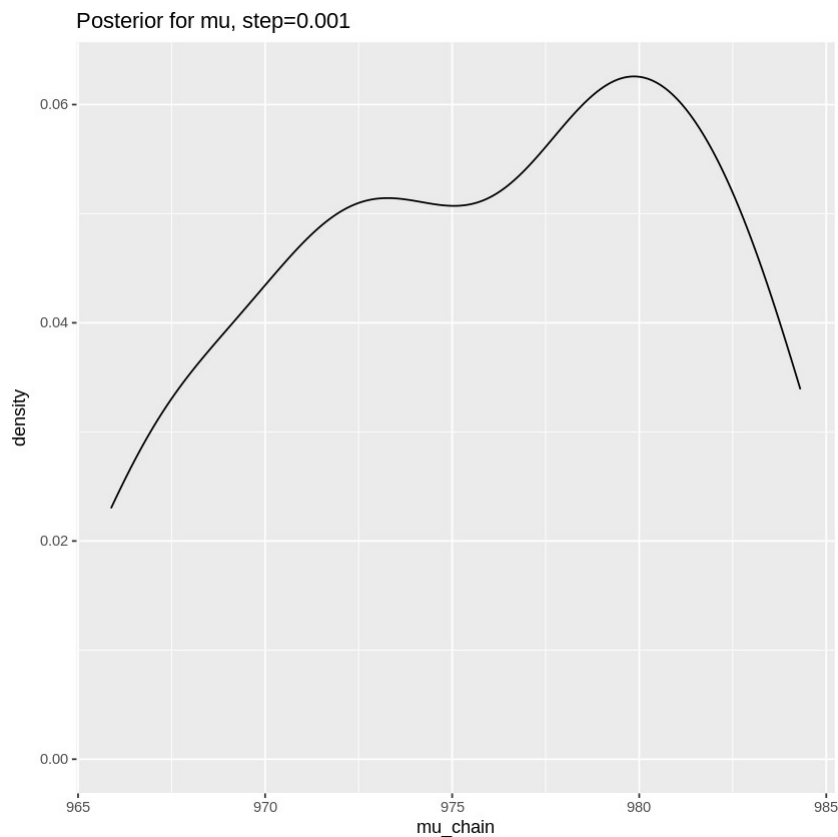
```

L=12, # no. of leapfrog steps
initial_q=c(1000,11), # Chain initialization
nsamp=6000, # total number of samples
nburn=2000) # number of burn-in samples

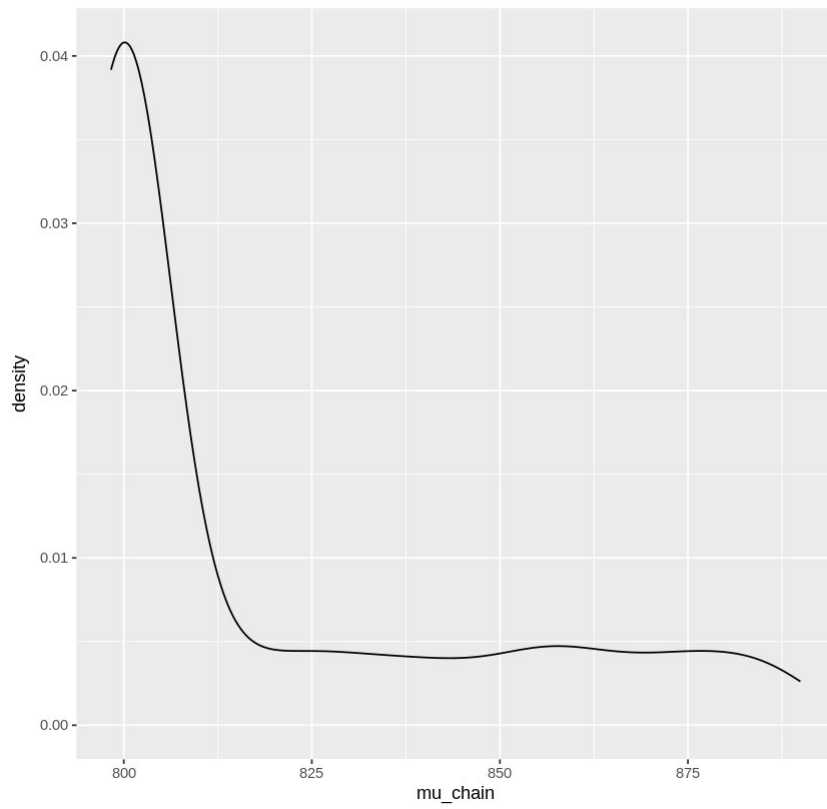
ggplot(df_100, aes(x=mu_chain)) + geom_density() + ggtitle("Posterior
for mu, step=0.001")
ggplot(df_1000, aes(x=mu_chain)) + geom_density() + ggtitle("Posterior
for mu, step=0.005")
ggplot(df_6000, aes(x=mu_chain)) + geom_density() + ggtitle("Posterior
for mu, step=0.02")

ggplot(df_100, aes(x=sigma_chain)) + geom_density() +
ggtitle("Posterior for sigma, step=0.001")
ggplot(df_1000, aes(x=sigma_chain)) + geom_density() +
ggtitle("Posterior for sigma, step=0.005")
ggplot(df_6000, aes(x=sigma_chain)) + geom_density() +
ggtitle("Posterior for sigma, step=0.02")

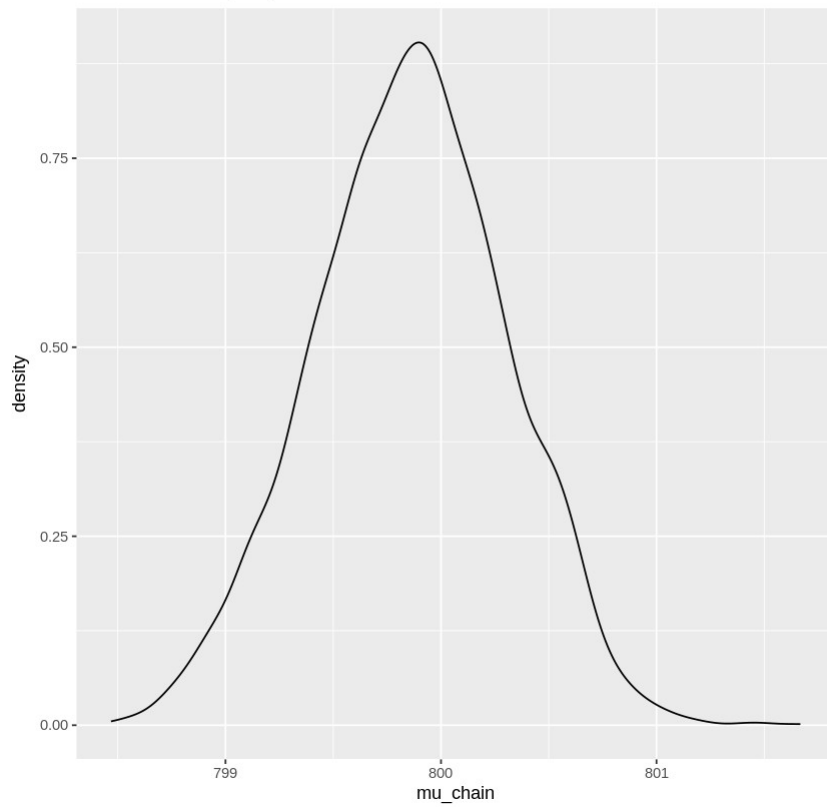
```



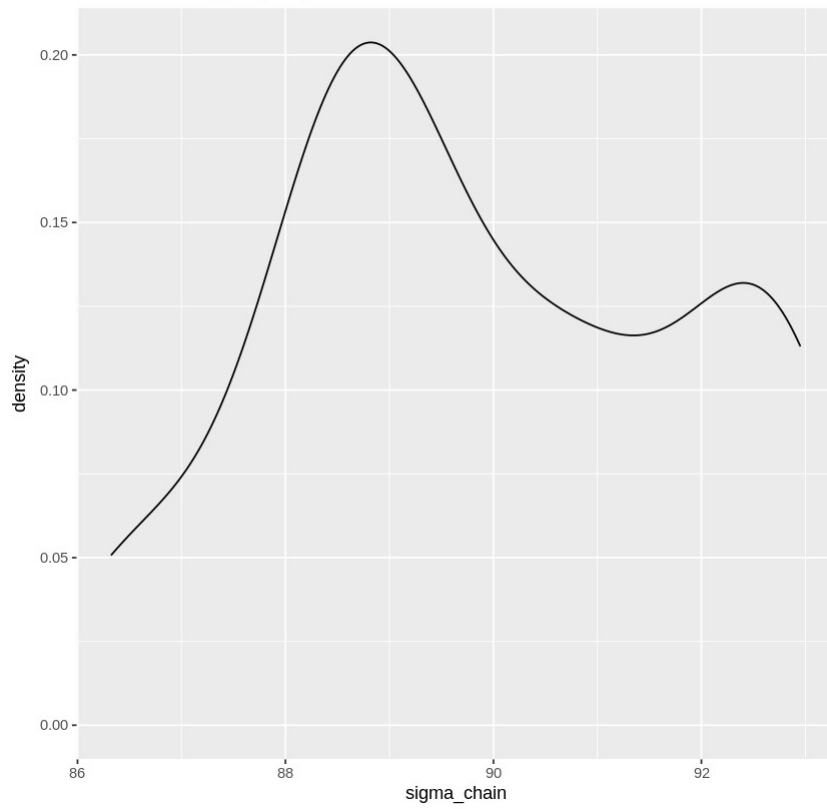
Posterior for μ , step=0.005



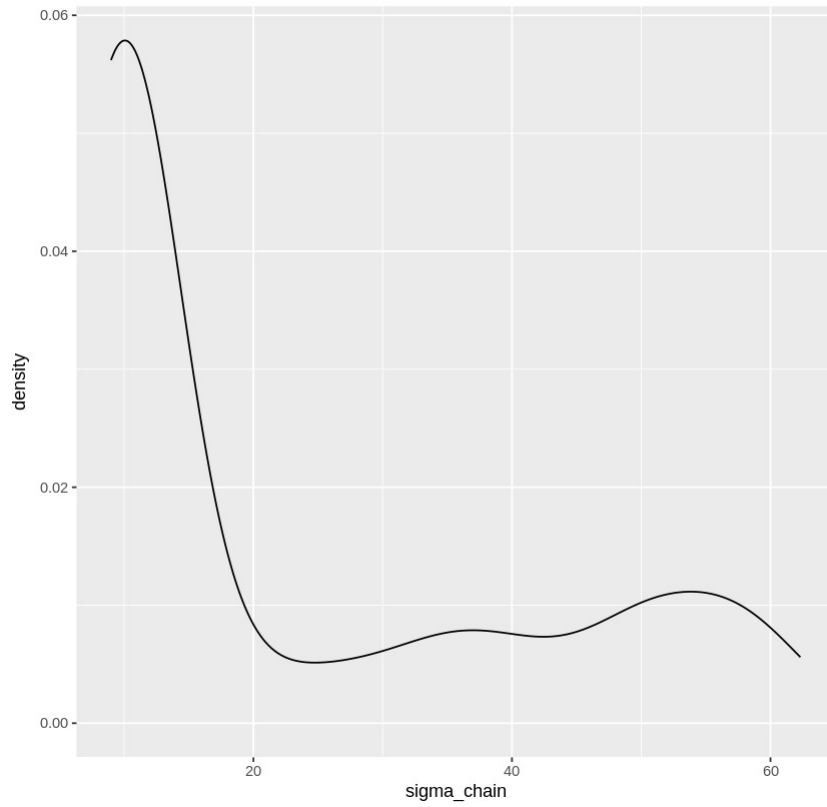
Posterior for μ , step=0.02

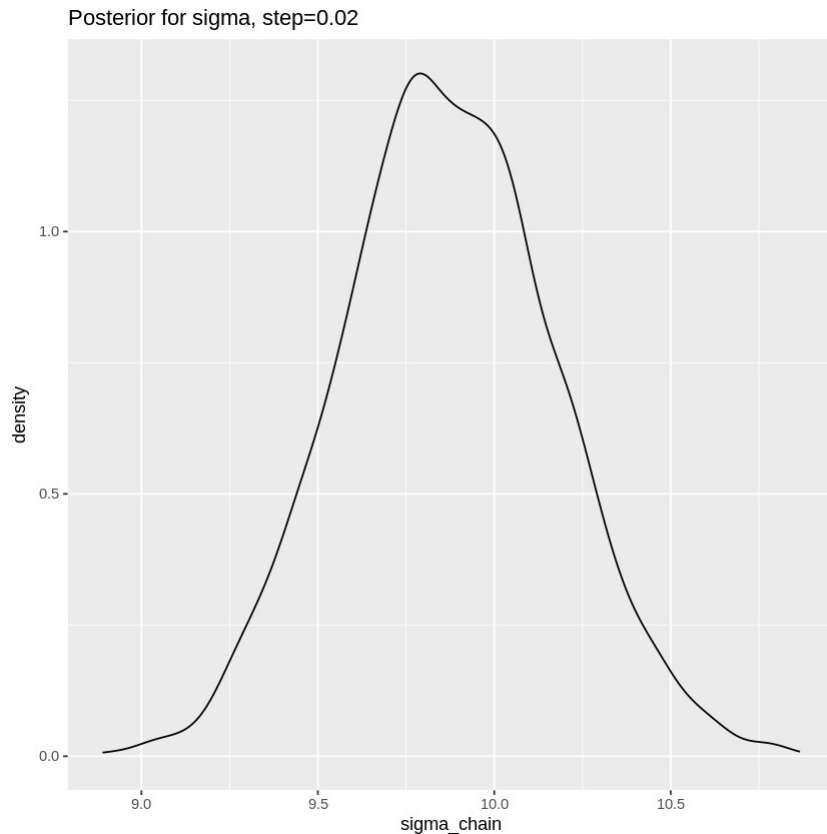


Posterior for sigma, step=0.001



Posterior for sigma, step=0.005





The step size dictates the trajectory of the simulation. If you inspect the trace plots, $\text{step} = 0.001$ exhibits extremely slow mixing and high autocorrelation (the chain barely moves because the momentum step is too minuscule). Conversely, if the step size gets too large (e.g., beyond 0.02), we risk the leapfrog integrator accumulating huge errors, and the chain to get stuck rejecting most proposals.

Exercise 2.4

After visually inspecting the μ and σ chains when we change the step-size, I observe that the problematic plot is that of $\text{step}=0.001$, because it has high autocorrelation. Each new sample is highly dependent on the previous sample, and takes very long to traverse the entire parameter space.

As for the plot with $\text{step}=0.02$, this looks like a normal posterior density distribution graph. It shows that the sampler is taking appropriately sized steps, mixing properly and exploring the high-probability regions efficiently.

Exercise 2.5

```
hmc_prior <- function(m_val, s_val) {
  df <- HMC(y=y, n=length(y), m=m_val, s=s_val, a=10, b=2,
    step=0.02, L=12, initial_q=c(1000,11),
    nsamp=6000, nburn=2000)
  df$prior <- paste("m=", m_val, ", s=", s_val, sep="")
  return(df)
}
```

```

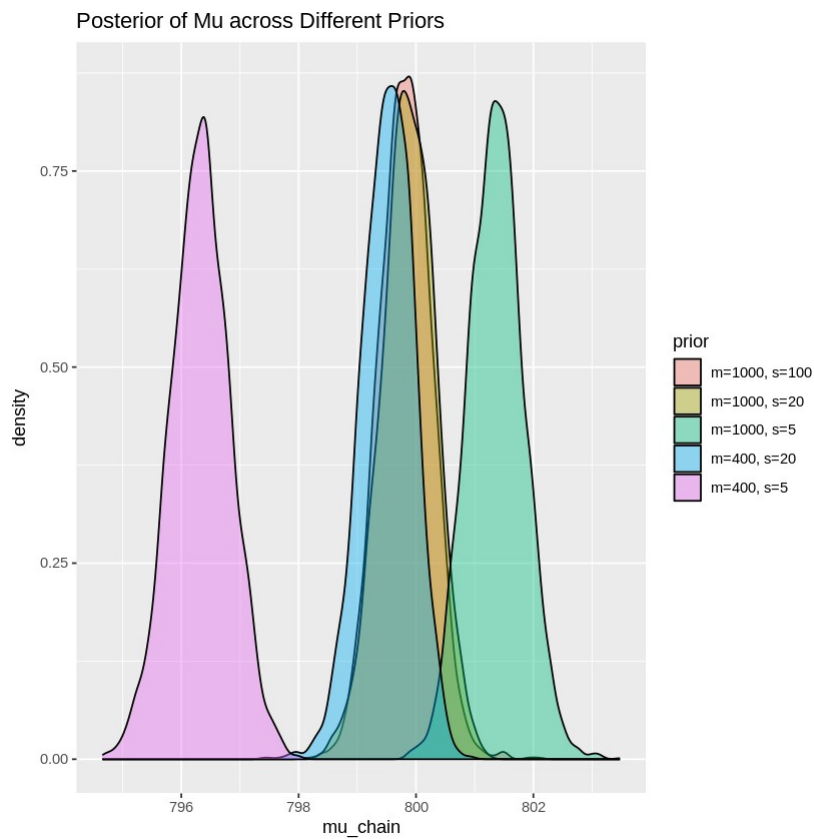
}

# Run models
df_prior1 <- hmc_prior(400, 5)
df_prior2 <- hmc_prior(400, 20)
df_prior3 <- hmc_prior(1000, 5)
df_prior4 <- hmc_prior(1000, 20)
df_prior5 <- hmc_prior(1000, 100)

df_prior_all <- rbind(df_prior1, df_prior2, df_prior3, df_prior4,
df_prior5)

ggplot(df_prior_all, aes(x=mu_chain, fill=prior)) +
  geom_density(alpha=0.4) +
  ggtitle("Posterior of Mu across Different Priors")

```



With 500 data points, the likelihood typically overwhelms the prior. However, highly restrictive priors (like $m=400, s=5$) will significantly pull and shift the posterior away from the actual data mean, whereas wide priors (like $m=1000, s=100$) will allow the posterior to naturally center around the true parameter, approximately 800.